
HOW WE DO IT

A guardrail is only as strong as the layer it is built into.

The usual pattern: write down what the AI must never do, put the rule in the AI's instructions, and call it a guardrail. This page is about the difference between a rule the AI has been asked to follow and one it cannot break.

[Talk to us about a guardrail review](#)[Read the Agent Authority Model →](#)

Asked, or unable

Almost every guardrail in production today is a sentence in a system prompt. It reads like a control, it is reported like a control, and under any determined attempt it behaves like a request.

The distinction is not academic. A rule the model has been asked to follow fails probabilistically — most of the time it holds, and the times it does not are the times someone was trying. A rule the model cannot break fails only if the thing enforcing it fails, and that thing can be tested like any other piece of software.

Everything below follows from one question: **where is the rule actually enforced?**

One rule, four different ways

The same rule can be written in any of four places. The choice decides whether someone determined can get past it — and how much work it takes to keep it true.

Policy

A rule in a document

A line in the AI usage policy. The system never sees it.

WHAT GETS PAST IT

Anyone who has not read it — and every automated path, which has nobody in it to read anything

1 of 4 ● ○ ○ ○

Prompt

A line in the AI’s instructions

The AI is told not to. The data is still in front of it.

WHAT GETS PAST IT

An attacker who can iterate. Against adaptive attack, instruction-level defences fail more often than they hold.

2 of 4 ● ● ○ ○

----- ABOVE THIS LINE YOU ARE ASKING · BELOW IT YOU ARE NOT -----

Software outside the AI that checks what goes in and out

Any answer containing another customer's record is blocked before it leaves.

WHAT GETS PAST IT

Reliable against opportunistic misuse. Unreliable against a motivated attacker: character and encoding evasion has reached complete bypass, and unsafe responses rise sharply in lower-resourced languages.

3 of 4 ●●●○

Architecture

The AI is given only the data and tools the task needs

The search returns only the signed-in customer's own records.

WHAT GETS PAST IT

Only what the scope itself permits — which is why the scope, not the guardrail, becomes the thing to review.

4 of 4 ●●●●

Architecture is not about withholding. The AI still gets everything it needs to do the job — it just never gets more than that.

"Never reveal another customer's data."

LAYER

Policy

WHAT IT IS

A rule in a document

IN THIS EXAMPLE

A line in the AI usage policy. The system never sees it.

STRENGTH

1 of 4

LAYER

Prompt

WHAT IT IS

A line in the AI's instructions

IN THIS EXAMPLE

The AI is told not to. The other customer's data is still in front of it.

WHAT GETS PAST IT

An attacker who can iterate. Against adaptive attack, instruction-level defences fail more often than they hold.

STRENGTH

2 of 4

LAYER

Runtime

WHAT IT IS

Software outside the AI that checks what goes in and out

IN THIS EXAMPLE

Any answer containing another customer's record is blocked before it leaves.

WHAT GETS PAST IT

Reliable against opportunistic misuse; unreliable against a motivated attacker. Character and encoding evasion has reached complete bypass against commercial guardrails, and unsafe-response rates rise sharply in lower-resourced languages.

STRENGTH

3 of 4

WHAT IT IS

The AI is given only the data and tools the task needs

IN THIS EXAMPLE

The search returns only the signed-in customer's own records.

WHAT GETS PAST IT

Only what the scope itself permits — which is why the scope, not the guardrail, becomes the thing to review.

STRENGTH

4 of 4

Architecture is not about withholding. The AI still gets everything it needs to do the job — it just never gets more than that.

Layers one and two ask the AI to behave. Layers three and four do not ask.
Stop at the prompt and your guardrails are requests, not controls.

5

THE FIFTH THING, WHICH IS NOT A LAYER

The model's own safety training is a real control, and it is the one you inherit rather than hold. That makes model selection a guardrail decision, and it is the reason two systems with identical guardrails do not carry identical risk.

The stopping rule

"Build it into the strongest layer you can" is advice. It never says *enough*, so it cannot close a design review, and it sends budget to the wrong places. The required strength follows from what it would cost to be wrong — the same exposure rating used in the Agent Authority Model.

enough

IF THE HANDOVER IS...

Undone at will, internal only

Nobody outside the organisation is affected

Prompt with monitoring

Reversible at a cost, one customer

Undoing it involves another party

Runtime a check outside the model, on the way in or out

Irreversible, one customer

Sent, paid, published or acted upon

Runtime + architectural scoping of the data and tools reached

Regulator-visible, public, or safety

A supervisor can inspect it, or anyone can see it

Architecture + an independent second control

Running above its exposure ceiling

More authority than the exposure alone permits

Architecture + a named artefact that carries the authority

Irreversible, one customer → Runtime, + architectural scoping of the data and tools reached

Regulator-visible, public, or safety → Architecture, + an independent second control

Running above its exposure ceiling → Architecture, + a named artefact that carries the authority

“The strongest layer you can” is advice. This is a decision — so a design review can close.

Exposure is rated exactly as it is in the Agent Authority Model: how hard the output is to undo, and who is exposed if it is wrong.

IF THE HANDOVER IS...

Trivially or windowed-reversible, internal only

ITS CONTROLS MUST BE ENFORCED AT LEAST AT...

Prompt, with monitoring

IF THE HANDOVER IS...

Reversible at a cost, one customer affected

ITS CONTROLS MUST BE ENFORCED AT LEAST AT...

Runtime

IF THE HANDOVER IS...

Irreversible, one customer affected

ITS CONTROLS MUST BE ENFORCED AT LEAST AT...

Runtime, plus architectural scoping of the data and tools it can reach

IF THE HANDOVER IS...

Architecture, plus an independent second control

IF THE HANDOVER IS...

Running above its exposure ceiling

ITS CONTROLS MUST BE ENFORCED AT LEAST AT...

Architecture, and the design must name the artefact that carries the authority

Now a design review can close. A client can be told they are done.

What you need to know about a control

The layer answers only one of the three things that matter. A framework that stops there has no place to put human approval, identity, or anything you would only discover afterwards.

Policy, prompt, runtime, architecture. This decides whether it can be broken, and how often it has to be re-tested.

Who is present when it runs?

In the loop, on the loop, out of the loop. Human approval is not a layer; it is an authority state. It is also a control with a capacity limit rather than an unlimited backstop — past the point where reviewers can attend properly, escalating more makes a system less safe, not more.

What would we know afterwards?

What is logged, what is monitored, what trips a circuit breaker, and how the output is undone. A blocking layer that produces no record fails silently. The most consequential documented AI exfiltration incident to date generated no logs, no alerts and no signatures.

Set, Prove & Hold

Three things happen to every guardrail. Four steps in each.

SET

BEFORE YOU BUILD

- 01 Name the handovers, not the rules.** Every moment the AI passes something to a person or another system. A "single agent" is usually four to seven of them.
- 02 Read the minimum enforcement layer off the exposure.** Build at or above that line.
- 03 Say what each control does — restrict, or carry the authority.** Most controls make a risky thing less risky. A few carry the authority themselves, and only those let a handover run above its ceiling.
- 04 Every control inherits the owner of the handover it defends.** No orphaned rules, no second register.

PROVE

BEFORE YOU LAUNCH

- 01 Attack each control directly, using a published corpus rather than imagination.**
- 02 Widen the attack to the whole system — including encoding and character-level evasion, and every language you operate in.**



Count what it blocks wrongly, with a denominator. What it catches and what it misses are two numbers, reported separately.

- 04 Record the result with a date, a version and a name.** Above the reversible-at-a-cost band, that name is not the person who built it.

HOLD

EVERY DAY AFTER

- 01 Watch the blocked attempts — and the unblocked ones.** What nothing fired on is the more interesting number.
- 02 Re-test on every change** to the model, the instructions, the scanner, the languages or the scope — and whenever a new attack class is published.
- 03 Add new controls, retire old ones, and move the authority.** Promotion on evidence; demotion automatic on incident.
- 04 Report enforcement strength weighted by exposure,** not a count.

What it costs to keep it true

LAYER

Policy

SET — HOW YOU BUILD IT

Write it in a document

PROVE — HOW YOU TEST IT

Nothing to test

HOLD — WHEN YOU RE-TEST

Never — it is not connected to anything

LAYER

Prompt

SET — HOW YOU BUILD IT

Put it in the AI's instructions

PROVE — HOW YOU TEST IT

Attack it. Expect it to fail some of the time.

HOLD — WHEN YOU RE-TEST

Every model or instruction change — so, constantly

LAYER

Runtime

SET — HOW YOU BUILD IT

Add a check outside the AI, on the way in or out

PROVE — HOW YOU TEST IT

Attack it in every language you serve, with encoding evasion, and count what it blocks wrongly

LAYER

Architecture

SET — HOW YOU BUILD IT

Give the AI only the data and tools the task needs

PROVE — HOW YOU TEST IT

Check the scoping is in place, then try to misuse what is inside the scope

HOLD — WHEN YOU RE-TEST

Whenever the scope changes — every tool, integration, credential or allowlist entry — and patch it like any other software

Read the last two columns together. A rule in the AI's instructions is quick to write and then needs re-testing every time anything moves. The same rule built into the architecture is real work once, and then holds until the scope drifts — which it does, quietly, every time someone adds a tool. Layer choice is a maintenance decision as much as a security one, and the maintenance never reaches zero.

WHAT TO MEASURE

Of the handovers that are irreversible, or visible to a regulator or the public — what share have their controls enforced at runtime or architecture?

Not how many guardrails you have. Not even how many sit at each layer: forty low-consequence rules moved to runtime make a beautiful chart and change nothing. Set assigns the layer. Prove tells you whether the claimed layer is real. Hold reports it over time.

Authority and enforcement

These are two halves of one instrument, and neither works alone.

The **Agent Authority Model** decides what a handover is allowed to do — capped by what it would cost to be wrong, raised only on measured evidence. This framework decides how strongly that is enforced, and how you know it still holds.

Authority is the *how much*. Guardrails are the *how*.

That is why a guardrail here is never a free-standing object with its own owner and its own register. It is a control attached to a handover, and it inherits that handover's owner — one register, one set of owners, one artefact a supervisor can inspect.

An authority model with unenforceable controls is a register of good intentions. Controls without an authority model are a pile of features nobody can justify to a regulator. Each is load-bearing for the other.

RELATED METHODOLOGY ↗

The Agent Authority Model →

Who authorised the agent to do that, and what must it prove to do more.

What this is built from

Nothing invented here, including the layering — it compresses a consensus already published by the major labs, the standards bodies and the academic literature. Two things are ours: making the re-test cadence a function of the layer, and binding required enforcement strength to exposure.

The ethics frameworks tell you what to forbid. The security ones tell you how it gets bypassed. A policy built on the first alone will name the right rules and miss every way around them.

WHAT TO FORBID

OWASP GenAI LLM Top 10 2026 · OWASP Top 10 for Agentic Applications · EU AI Act Article 5, as amended · national law on unlawful and manipulated content in the markets you serve

HOW IT GETS BYPASSED, AND HOW TO TEST

MITRE ATLAS v2026.08 · NIST AI 100-2 E2025, Adversarial Machine Learning · OWASP AI Testing Guide v1.0 · UK AI Security Institute, Principles for Evaluating Misuse Safeguards

WHAT YOU WILL BE MEASURED AGAINST

ISO/IEC 42001:2023 with ISO/IEC 42006:2025 · NIST AI RMF 1.0 and AI 600-1 · CSA AI Controls Matrix v1.1 · the data protection and sector regulation of the jurisdiction you operate in · the UAE Charter for AI

Three moves

- 1 Move 1 — an afternoon.** List every handover — every moment the AI passes something to a person or another system — and mark which layer each one's controls are enforced at. Then look only at the irreversible and the regulator-visible. That number is the finding.
- 2 Move 2 — an afternoon, two people.** Take the three that matter most and try to break them, in every language the service operates in. Whatever survives is worth keeping; whatever does not was never a control.
- 3 Move 3.** Move one guardrail to a stronger layer. Start with the least technical option: narrow what the AI is given to what the task actually needs.

Most guardrail reviews find the same thing.

That the controls protecting the highest-consequence handovers are the ones nobody can test. Finding out takes an afternoon.



Book a guardrail review



Intelligence that moves work.

Market view · United Arab Emirates

CAPABILITY

What we do

Platforms

Industries

IDAO methodology

Agent Authority Model

COMPANY

About & Leadership

Insights

Partners

Contact

ACTION



Bring us one process

